

WORLD KARATE · BACKEND

مستندات بک‌اند World Karate

راهنمای جامع معماری، API، داده، امنیت و
استقرار

راهنمای معماری، API، مدل داده، اجرا و استقرار نسخه سند، ۱ * تاریخ بررسی کد ۲۳ ژوئن ۲۰۲۶ * نسخه برنامه 1.0.0

بک‌اند WorldKarate سرویس سمت سرور سامانه دوره‌های آموزشی کاراته است. این برنامه یک API مبتنی بر JSON برای ثبت‌نام و ورود، بازیابی رمز عبور، ویرایش پروفایل، نمایش دوره‌ها، مدیریت دوره توسط ادمین، دسترسی به دوره‌های خریداری‌شده، عضویت در خبرنامه و پرداخت تأیید تراکنش از طریق زرین‌پال فراهم می‌کند.

این سند رفتار واقعی نسخه فعلی کد را شرح می‌دهد. مواردی که در مدل داده یا وابستگی‌ها دیده می‌شوند اما هنوز کامل پیاده‌سازی نشده‌اند، صریحاً در بخش محدودیت‌ها و گام‌های بعدی آمده‌اند.

فهرست مطالب

۱. نمای کلی
۲. راه‌اندازی سریع
۳. تنظیم متغیرهای محیطی
۴. معماری
۵. مدل داده
۶. احراز هویت و مجوزها
۷. رفتار قابلیت‌ها
۸. مرجع API
۹. چرخه پرداخت
۱۰. اعتبارسنجی و خطاها
۱۱. امنیت
۱۲. ساخت، استقرار و عملیات
۱۳. آزمون و رفع اشکال
۱۴. محدودیت‌ها و گام‌های بعدی

نمای کلی

بخش	پیاده‌سازی
محیط اجرا	Node.js، فریم‌ورک Express، TypeScript و خروجی CommonJS
پایگاه داده	PostgreSQL از طریق Prisma
احراز هویت	JWT در کوکی HTTP-only با نام auth-token
رمز عبور	bcrypt با هزینه قابل تنظیم
اعتبارسنجی	Zod
پرداخت	SDK زرین‌پال؛ حالت Sandbox در توسعه
سخت‌سازی HTTP	Helmet، CORS، محدودیت حجم بدنه و cookie-parser
موجودیت‌ها	User، Course، Transaction، Newsletter و دو جدول واسط
سیک API	endpointهای JSON با پیام‌های کاربرمحور فارسی
تست خودکار	در نسخه فعلی وجود ندارد

قابلیت‌های پیاده‌سازی‌شده

- ساخت حساب با ایمیل یکتا و رمز هش‌شده با bcrypt
- ورود با کوکی JWT و خروج با خالی‌کردن همان کوکی؛
- آغاز بازیابی رمز، بررسی OTP چهاررقمی و تعیین رمز جدید؛
- ویرایش نام و ایمیل کاربر و صدور JWT جدید؛
- مشاهده عمومی فهرست دوره‌ها و جزئیات یک دوره؛

- ایجاد/حذف دوره و جست‌وجوی دوره‌های کاربر توسط ادمین؛
- مشاهده دوره‌های خریداری‌شده کاربر واردشده؛
- ثبت ایمیل یکتا در خبرنامه؛
- محاسبه سبد با قیمت‌های سمت سرور، ساخت تراکنش زرین‌پال، تأیید پرداخت و اعطای دسترسی به دوره‌ها به‌شکل `idempotent`؛

راه‌اندازی سریع

پیش‌نیازها

- Node.js نسخه ۲۰ یا بالاتر پیشنهاد می‌شود؛ هنگام تولید این سند پروژه با Node 22.17.1 و npm 11.16.0 با موفقیت `build` شد؛
- یک `PostgreSQL` قابل دسترسی با `connectionString` معتبر؛
- شناسه پذیرنده و `accessToken` زرین‌پال؛ این دو در زمان بالا آمدن برنامه بررسی می‌شوند، حتی اگر `endpoint` پرداخت فراخوانی نشود؛

۱ نصب وابستگی‌ها

```
npm install
```

اسکرپت `postinstall` به صورت خودکار `prisma generate` را اجرا می‌کند؛

۲ ساخت فایل محیطی

در ریشه پروژه فایل `env` بسازید؛ مقادیر واقعی `secret` را `commit` نکنید؛

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST:5432/DATABASE?schema=public"
JWT_SECRET="replace-with-a-long-random-secret"
PORT=3000
ROUNDS=10
ZARINPAL_MERCHANT_ID="your-merchant-id"
ZARINPAL_ACCESS_TOKEN="your-access-token"
FRONTEND_URL="http://localhost:5173"
BACKEND_URL="http://localhost:3000"
```

مقدار `ENV_MODE` توسط اسکرپت‌های `npm` تعیین می‌شود و معمولاً نباید داخل `env` نوشته شود؛

۳ آماده‌سازی پایگاه داده

در محیط توسعه؛

```
npx prisma migrate dev
npx prisma generate
```

در استقرار، فقط `migration`های ثبت‌شده را اعمال کنید؛

```
npx prisma migrate deploy
```

۴ اجرای توسعه

```
npm run dev
```

`Nodemort` فایل `app.ts` را با `ENV_MODE=development` اجرا می‌کند؛ آدرس پیش‌فرض `API` برابر `http://localhost:3000` است؛

۵ `build` و اجرای `production`

```
npm run build
npm start
```

خروجی `JavaScript` و `source map` در `dist/` قرار می‌گیرد و دستور `start` فایل `dist/app.js` را با حالت `production` اجرا می‌کند؛

۶ تعیین نخستین ادمین

endpoint برای ساخت ادمین وجود ندارد؛ ابتدا کاربر عادی ثبت کنید، سپس از یک فرایند مدیریتی کنترل‌شده استفاده کنید؛

```
UPDATE "User" SET "isAdmin" = true WHERE email = 'admin@example.com';
```

تنظیم متغیرهای محیطی

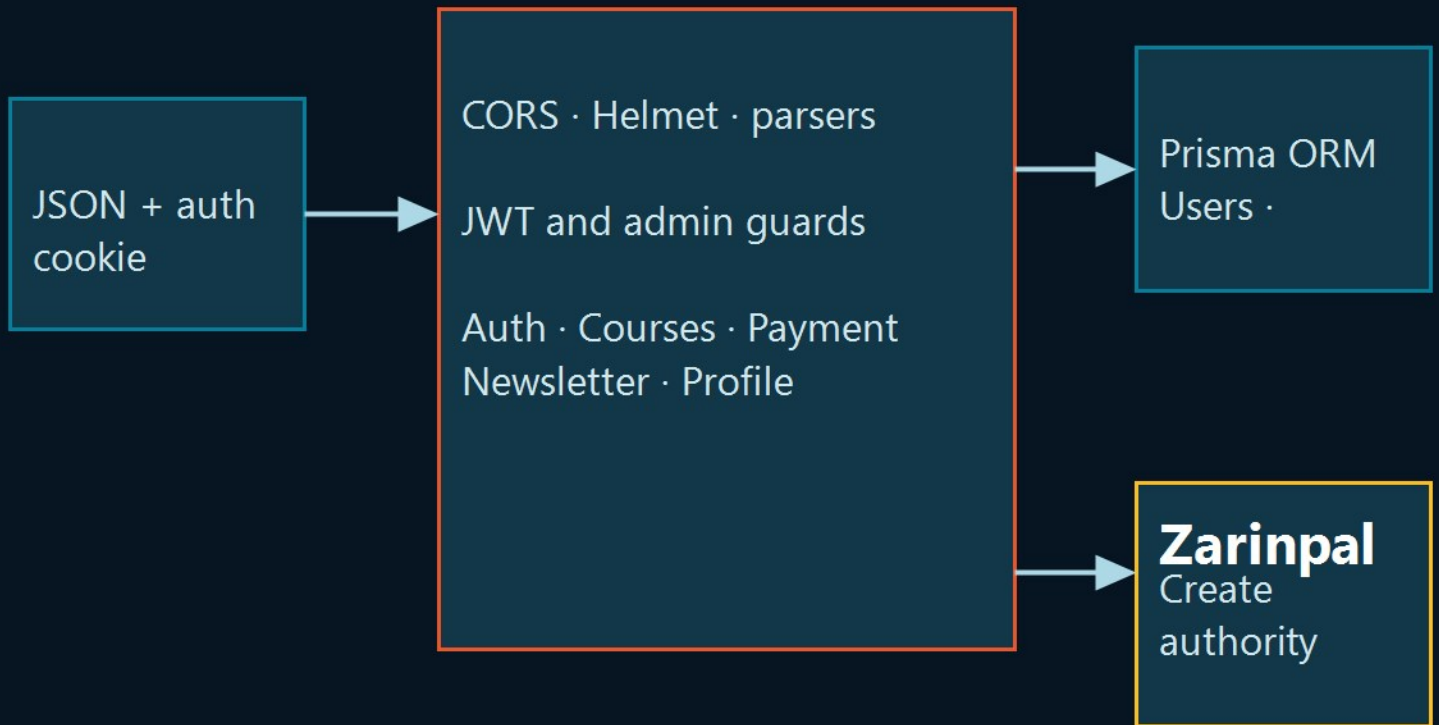
متغیر	الزامی	کاربرد	رفتار پیش‌فرض
DATABASE_URL	بله	اتصال PostgreSQL برای Prisma	بدون پیش‌فرض
JWT_SECRET	بله	امضا و بررسی JWT	بدون پیش‌فرض امن
ZARINPAL_MERCHANT_ID	بله	شناسه پذیرنده زرین‌پال	در صورت نبود، startup متوقف می‌شود
ZARINPAL_ACCESS_TOKEN	بله	احراز درخواست‌های SDK زرین‌پال	در صورت نبود، startup متوقف می‌شود
PORT	خیر	پورت Express	3000
ROUNDS	خیر	work factor مربوط به bcrypt	10
FRONTEND_URL	خیر	مبنای callback به /payment/verify	http://localhost:3000
BACKEND_URL	خیر	در قرارداد فایل محیطی محلی وجود دارد	فعلاً در کد استفاده نشده
ENV_MODE	از طریق scripts	انتخاب تنظیمات development/production	توسط scripts تنظیم می‌شود

در production؛

- CORS فقط `https://worldkarate.ir` و `https://www.worldkarate.ir` را با `credentials` می‌پذیرد؛
 - کوکی `Secure` می‌شود، `SameSite=Lax` باقی می‌ماند و `domain` آن `worldkarate.ir` است؛
 - زرین‌پال در حالت واقعی و با آدرس `https://www.zarinpal.com/pg/StartPay` استفاده می‌شود؛
- در `development`، `origin` بازتاب داده می‌شود (`origin:true`)، کوکی `Secure` نیست و درگاه `sandbox` زرین‌پال انتخاب می‌شود؛

معماری

Runtime Architecture



Database state is authoritative for identity, roles, prices, and course

برنامه یک `modular/monolith` تک‌پردازه است. فایل `app.ts/middleware`های مشترک را تنظیم می‌کند و `router`های کوچک `Express` را `mount` می‌کند. هر `router` مستقیماً با `Prisma` کار می‌کند و لایه `service/repository` جداگانه‌ای وجود ندارد. این طراحی پروژه را کوچک و قابل دنبال کردن نگه داشته، اما قواعد مشترک و مرز تراکنش‌های دیتابیس باید در هر `route` مدیریت شوند.

ساختار سورس

مسیر	مسئولیت
<code>app.ts</code>	ترکیب برنامه، <code>middleware</code> های مشترک و <code>mount</code> مسیرها
<code>middleware/authorization.ts</code>	بررسی کوکی <code>JWT</code> و بارگذاری کاربر
<code>middleware/adminAuth.ts</code>	کنترل نقش ادمین بر اساس دیتابیس
<code>prisma/schema.prisma</code>	مدل مرجع داده
<code>prisma/migrations/</code>	تاریخچه <code>migrations</code> های <code>PostgreSQL</code>
<code>prisma/db.ts</code>	نمونه مشترک <code>PrismaClient</code>
<code>schemas/</code>	<code>Schema</code> های اعتبارسنجی <code>Zod</code>
<code>src/auth/</code>	ثبت نام، ورود، بازیابی، پروفایل و خروج
<code>src/courses/</code>	مسیرهای عمومی، کاربر و ادمین دوره
<code>src/newsletter/</code>	ثبت مشترک خبرنامه
<code>src/payment/</code>	<code>checkout</code> و تأیید زرین پال
<code>utils/</code>	ابزارهای <code>JWT</code> ، <code>cookie</code> ، <code>environment</code> ، <code>OTP</code> و <code>SDK</code>
<code>dist/</code>	<code>JavaScript</code> کامپایل شده

خط لوله پردازش هر درخواست

تمام درخواست‌ها به ترتیب از `parser`، `CORS`، بدنه `Helmet`، `JSON/urlencoded` و `cookie` عبور می‌کنند. `endpoint`های محافظت شده سپس `middleware/authorization` را اجرا می‌کنند.

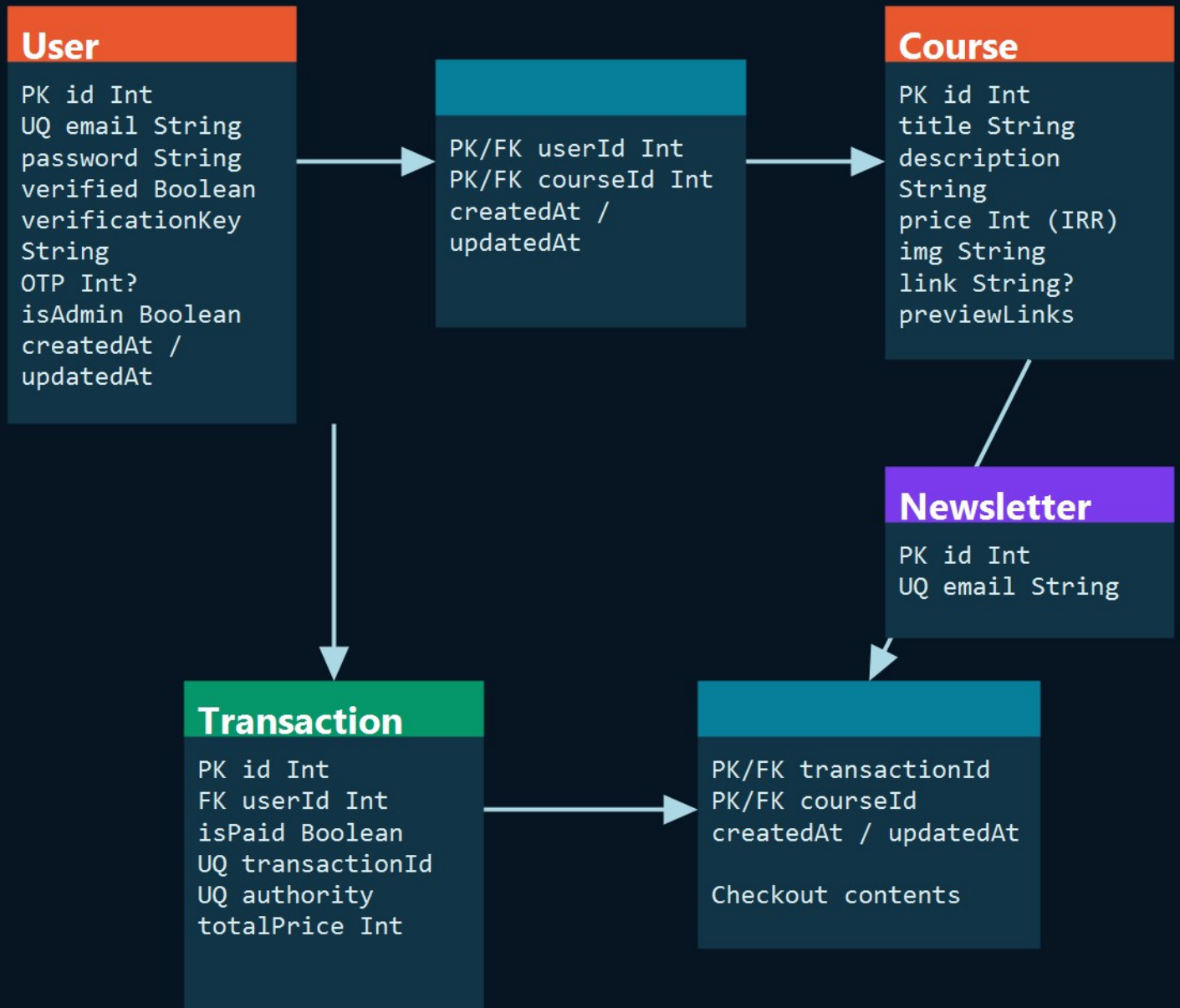
۱. خواندن `auth-token` از `cookie`
 ۲. بررسی امضا با `JWT_SECRET`
 ۳. یافتن کاربر فعلی با ایمیل موجود در `token`
 ۴. قرار دادن رکورد کامل کاربر در `req.body.user`
 ۵. ادامه `router` و، برای مسیرهای ادمین، خواندن مجدد `isAdmin` از دیتابیس.
- به این ترتیب دیتابیس مرجع نهایی هویت و نقش است و `client` نمی‌تواند با تغییر `body` یا یک `claim` قدیمی خود را ادمین کند.

مرزهای بیرونی

- `PostgreSQL` تمام وضعیت پایدار برنامه را ذخیره می‌کند.
- زرین پال `authority` پرداخت را صادر و پرداخت نهایی را تأیید می‌کند.
- فرانت‌اند کاربر را به صفحه درگاه هدایت می‌کند و پس از بازگشت، `authority` را به `endpoint` تأیید ارسال می‌کند.
- ارسال ایمیل فعلاً متصل نیست. پکیج `Nodemailer` نصب شده، اما هیچ `mailer`ی فراخوانی نمی‌شود.

مدل داده

Entity Relationship Diagram



مسئولیت موجودیت‌ها

موجودیت	مسئولیت و محدودیت مهم
User	هویت، hash، رمز، نقش ادمین، OTP و داده‌های Verification، ایمیل و verification key یک‌هستند.
Course	رکورد کاتالوگ، قیمت عدد صحیح و بر حسب ریال است link، در دیتابیس nullable و previewLinks آرایه text است.
Transaction	یک تلاش checkout متعلق به یک کاربر، شامل مبلغ کل، شناسه‌های پرداخت و وضعیت پرداخت.
UsersOnCourses	رابطه مالکیت با کلید مرکب؛ هر کاربر هر دوره را حداکثر یک‌بار دارد.
TransactionsOnCourses	دوره‌های داخل یک تلاش checkout با کلید مرکب.
Newsletter	ایمیل یکنای خبرنامه و مستقل از User.

معنای رابطه‌ها

- هر کاربر صفر یا چند تراکنش دارد؛
- رابطه User و Course از طریق UsersOnCourses چندبه‌چند است؛
- رابطه Transaction و Course از طریق TransactionsOnCourses چندبه‌چند است؛
- رفتار حذف foreign key restrictive است؛ بنابراین حذف دوره‌ای که در مالکیت یا تراکنش استفاده شده ممکن است شکست بخورد؛
- join مربوط به تراکنش، محتوای سید را نگه می‌دارد و totalPrice مبلغ زمان خرید را حتی پس از تغییر قیمت دوره حفظ می‌کند.

نکته شناسه‌های پرداخت

در checkout، هر دو فیلد authority و transactionId ابتدا authority زیرین پال را نگه می‌دارند. پس از تأیید موفق، transactionId با ref_id جایگزین می‌شود ولی authority کلید پایدار جست‌وجو باقی می‌ماند. همچنین در response اولیه، نام transactionId برای ID عددی داخلی استفاده شده است.

- transactionId در checkout response کلید اصلی عددی دیتابیس؛
- Transaction.transactionId در مدل authority خارجی پیش از پرداخت و ref_id پس از پرداخت.

احراز هویت و مجوزها

کوکی و JWT

نام کوکی auth-token، نوع آن HTTP-only، مسیر آن / و عمر آن ۱۰ روز است. JavaScript مرورگر نمی‌تواند آن را بخواند. درخواست‌های فرانت‌اند باید credentials را ارسال کنند؛ برای نمونه:

```
fetch(url, { credentials: "include" })
```

JWT شامل email، firstName، lastName و isAdmin است. token فعلاً expiration صریح ندارد و ماندگاری مرورگر عملاً با عمر cookie کنترل می‌شود. پس از ویرایش پروفایل JWT جدید ساخته می‌شود.

ثبت‌نام و ورود

ثبت‌نام نام، ایمیل و رمز را بررسی می‌کند، یکنایی ایمیل را می‌سنجد، نام‌ها را lowercase کرده و حرف اول را بزرگ می‌کند، رمز را hash می‌کند و یک verification key تصادفی ۸۳ کاراکتری می‌سازد. ثبت‌نام کاربر را خودکار login نمی‌کند.

ورود برای ایمیل ناشناخته و رمز اشتباه یک پیام عمومی مشابه برمی‌گرداند. در موفقیت، cookie تنظیم و فیلدهای عمومی پروفایل برگردانده می‌شوند.

بازیابی رمز

رفتار فعلی چنین است:

۱. PUT /forget-password یک OTP چهاررقمی تولید و روی User ذخیره می‌کند و آن را در JSON برمی‌گرداند؛
۲. POST /validate-otp عدد ارسالی را مقایسه می‌کند و در موفقیت همان cookie ورود عادی را صادر می‌کند؛
۳. PUT /reset-password با آن cookie محافظت می‌شود و رمز جدید مطابق با hash و ذخیره می‌کند.

OTP فعلاً ایمیل نمی‌شود، expiration ندارد، تک‌مصرف نیست، rate limit ندارد و پس از استفاده پاک نمی‌شود؛ بنابراین تا اجرای اصلاحات بخش محدودیت‌ها باید این قابلیت را در مرحله توسعه در نظر گرفت؛

ماتریس دسترسی

قابلیت	عمومی	کاربر واردشده	ادمین
ثبت‌نام، ورود، بازیابی و خیرنامه	✓	✓	✓
فهرست و جزئیات دوره	✓	✓	✓
checkout و تأیید	✓ verify عمومی؛ checkout نیارمند login	✓	✓
ویرایش پروفایل، خروج و دوره‌های خود		✓	✓
ایجاد و حذف دوره			✓
جست‌وجوی دوره‌ها با ایمیل کاربر			✓

رفتار قابلیت‌ها

دوره‌ها و دسترسی

همه می‌توانند فهرست دوره‌ها و یک دوره را ببینند؛ رکورد دوره شامل عنوان، توضیح، قیمت ربالی، تصویر، لینک کامل اختیاری در دیتابیس و لینک‌های preview است؛ endpoint مالکیت، تمام دوره‌هایی را برمی‌گرداند که relation آنها شامل کاربر فعلی است؛

endpoint ساخت دوره عنوان، توضیح، قیمت عددی غیرمنفی، تصویر و لینک را اعتبارسنجی می‌کند؛ با وجود nullable بودن Course.link در Prisma، API آن را اجباری می‌داند؛ previewLinks در route ساخت پشتیبانی نمی‌شود؛

مدیریت

کنترل ادمین به claim داخل JWT اعتماد نمی‌کند و نقش را از دیتابیس می‌خواند؛ endpoint های ادمین دوره می‌سازند، دوره را با ID حذف می‌کنند و تلاش می‌کنند دوره‌های متعلق به ایمیل مشخص را نمایش دهند؛

خبرنامه

عضویت، ایمیل معتبر و غیروابسته به User را ذخیره می‌کند و duplicate را رد می‌کند؛ endpoint مشاهده، لغو عضویت یا ارسال campaign وجود ندارد؛

پرداخت و اعطای دوره

checkout شناسه‌های دوره را دریافت می‌کند، دوره‌های واقعی و قیمت آنها را از دیتابیس می‌خواند، سید خالی/نامعتبر و دوره از قبل خریداری‌شده را رد می‌کند و مبلغ را صرفاً در سرور محاسبه می‌کند؛ پس از دریافت authority، تراکنش پرداخت‌نشده و ردیف‌های دوره ساخته می‌شوند و URL درگاه به client می‌رسد؛

تأیید پرداخت تراکنش را با authority پیدا می‌کند و مبلغ ذخیره‌شده را برای verification سرور به سرور می‌فرستد؛ کد 100 وضعیت را paid، ref_id را paid، ref_id را ذخیره و مالکیت دوره‌ها را با رد duplicate ایجاد می‌کند؛ تراکنش قبلاً paid نیز بدون اعطای دوباره با موفقیت پاسخ می‌دهد؛

مرجع API

تمام bodyها و responseها JSON هستند؛ پیام‌ها عمدتاً فارسی‌اند؛ خطاهای پیش‌بینی‌نشده معمولاً 500 هستند؛

فهرست endpointها

متد	مسیر	دسترسی	کاربرد
POST	/register	عمومی	ساخت حساب
POST	/login	عمومی	ورود و تنظیم cookie
POST	/logout	کاربر	پاک کردن cookie
PUT	/profile	کاربر	ویرایش پروفایل خود
PUT	/forget-password	عمومی	تولید OTP
POST	/validate-otp	عمومی	بررسی OTP و صدور cookie

PUT	/reset-password	کاربر	تعیین رمز جدید
GET	/fetch-course	عمومی	فهرست دوره‌ها
GET	/fetch-course/:courseId	عمومی	یک دوره
POST	/create-course	ادمین	ساخت دوره
DELETE	/delete-course/:courseId	ادمین	حذف دوره
GET	/admin/fetch-course/:email	ادمین	دوره‌های یک کاربر
GET	/user/fetch-course	کاربر	دوره‌های کاربر فعلی
POST	/register-newsletter	عمومی	عضویت خبرنامه
POST	/payment/checkout	کاربر	ساخت تراکنش
POST	/payment/verify	عمومی	تأیید authority و اعطای دوره

endpointهای هویت

POST /register

```
{
  "firstName": "Sara",
  "lastName": "Ahmadi",
  "email": "sara@example.com",
  "password": "karate123"
}
```

نام‌ها ۳ تا ۲۰ حرف/فاصله هستند؛ رمز حداقل ۸ کاراکتر و شامل حداقل یک حرف کوچک لاتین و یک رقم است؛ موفقیت 200 و verificationKey، ایمیل تکراری 409 و داده نامعتبر 400 می‌دهد؛

POST /login

```
{ "email": "sara@example.com", "password": "karate123" }
```

در موفقیت 200، کوکی auth-token و نام/نام خانوادگی/ایمیل عمومی کاربر برمی‌گردد؛ ورودی نامعتبر 400 و credential اشتباه 403 است؛

POST /logout

نیازمند cookie است و با همان گزینه‌های cookie مقدار آن را خالی می‌کند؛ پاسخ 200 است؛

PUT /profile

هر فیلد اختیاری است؛

```
{ "firstName": "سارا", "lastName": "احمدی", "email": "new@example.com" }
```

نام‌ها ۳ تا ۲۰ حرف فارسی یا لاتین و بدون فاصله‌اند؛ موفقیت 200 و JWT تازه صادر می‌کند؛ conflict؛ ایمیل فعلاً به 500 می‌رسد؛

PUT /forget-password

```
{ "email": "sara@example.com" }
```

در موفقیت OTP؛ message؛ با 200 برمی‌گردد؛ ایمیل نامعتبر 400 و ناشناخته 404 است؛ متن خطا برای جلوگیری از افشای حساب مشابه است؛

POST /validate-otp

```
{ "email": "sara@example.com", "OTP": 4831 }
```

OTP باید number بین ۱۰۰۰ تا ۹۹۹۹ باشد؛ موفقیت cookie می‌سازد و mismatch یا ورودی نامعتبر 400 است؛

PUT /reset-password

```
{ "newPassword": "newpass123", "repeatPassword": "newpass123" }
```

نیازمند cookie است؛ هر دو رمز باید با policy منطبق و با هم برابر باشند؛ پاسخ موفق 200 است؛

endpoint های دوره**GET /fetch-course**

آرایه همه دوره‌ها را با 200 می‌دهد؛ pagination، sort، filter یا projection وجود ندارد؛

GET /fetch-course/:courseId

یک دوره با 404 می‌دهد؛ ID عددی نامعتبر هنگام فراخوانی Prisma به 400 تبدیل می‌شود؛

POST /create-course

```
{
  "title": "Advanced Kumite",
  "description": "A complete advanced kumite training program.",
  "price": 2500000,
  "img": "https://cdn.example.com/kumite.jpg",
  "link": "https://courses.example.com/kumite"
}
```

نیازمند user و admin؛ عنوان ۵-۵، توضیح ۲۰-۱۰۰ و قیمت number غیرمنفی است؛ img و link اجباری ولی URL-validate نشده‌اند؛ موفقیت 200 و رکورد ساخته شده را می‌دهد؛

DELETE /delete-course/:courseId

فقط ادمین؛ موفقیت 200؛ ID ناشناخته/نامعتبر یا مانع foreign key فعلاً 500 می‌شود؛

GET /user/fetch-course

تمام رکوردهای دوره متعلق به کاربر واردشده را می‌دهد؛

GET /admin/fetch-course/:email

فقط ادمین؛ کاربر ناشناخته 400 است؛ ایراد فعلی؛ کد از join table فیلد userId را به جای courseId می‌کند؛ نتیجه ممکن است اشتباه باشد؛

خبرنامه**POST /register-newsletter**

```
{ "email": "reader@example.com" }
```

موفقیت 200 و ایمیل نامعتبر یا تکراری 400 است؛

پرداخت**POST /payment/checkout**

```
{ "courseIds": ["1", "2"] }
```

route انتظار آرایه دارد و هر مقدار را با Number تبدیل می‌کند؛ schema زاد ندارد و مقدار null حذف شده type اشتباه می‌تواند به خطای سرور برسد؛ اگر حداقل یک ID معتبر باشد، IDهای ناموجود بی‌صدا کنار گذاشته می‌شوند؛ duplicateها نیز با query دیتابیس ادغام می‌شوند؛

نمونه پاسخ موفق؛

```
{
  "message": "در حال انتقال به درگاه پرداخت",
  "paymentUrl": "https://sandbox.zarinpal.com/pg/StartPay/A000...",
  "authority": "A000...",
  "transactionId": 42
}
```

سید خالی، کاملاً نامعتبر، دارای دوره قبلاً خریداری‌شده یا درخواست ردشده gateway پاسخ 400 می‌دهد.

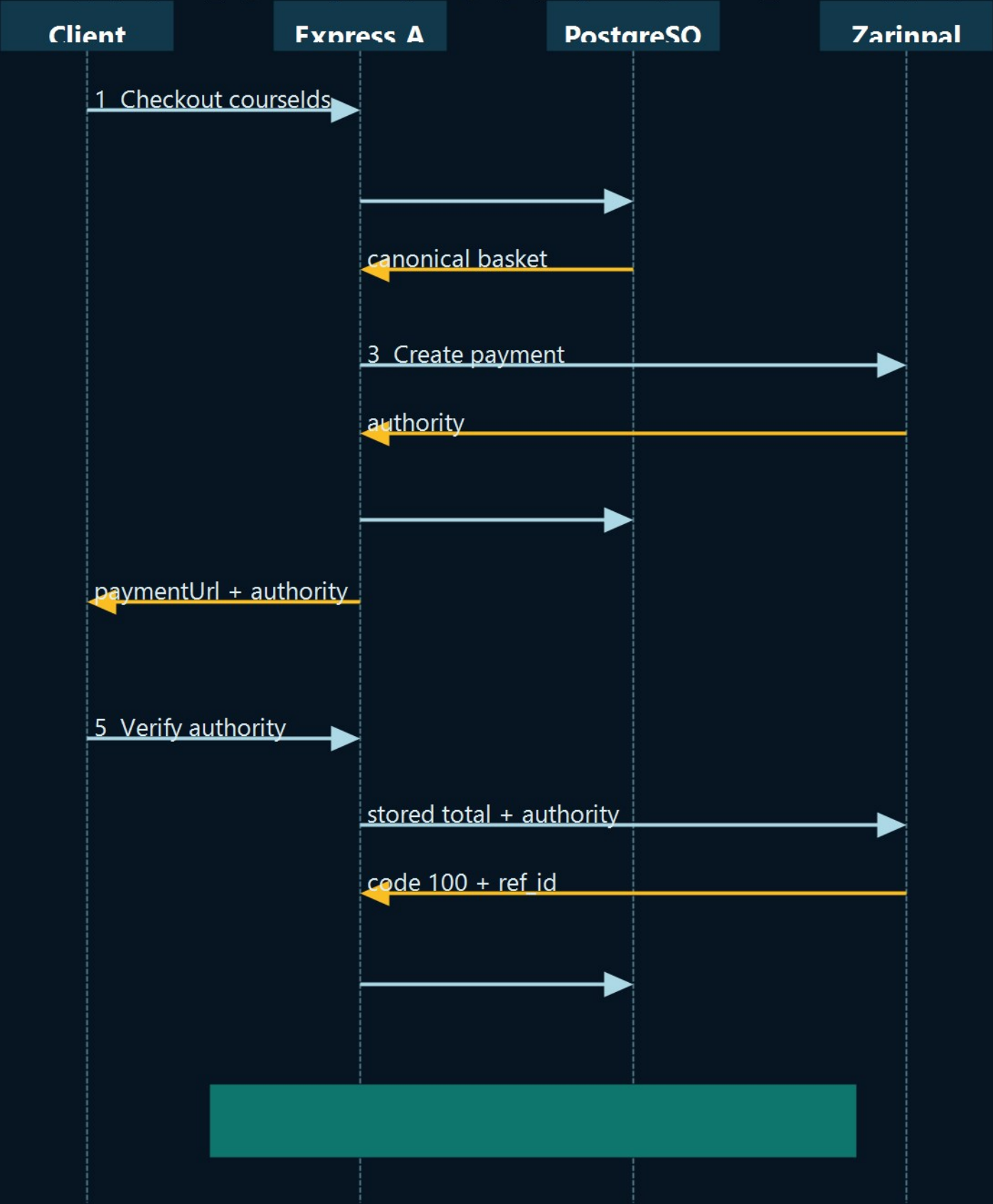
POST /payment/verify

```
{ "authority": "A000..." }
```

تراکنش ناموجود 404 است. کد 100 زرین‌پال پاسخ 200 می‌دهد، تراکنش را paid می‌کند و دوره‌ها را اعطا می‌کند. تراکنش قبلاً paid نیز 200 است. نتیجه ناموفق gateway پاسخ 400 دارد route عمومی است چون endpoint تکمیل پرداخت محسوب می‌شود و authority معتبر تراکنش را مشخص می‌کند.

چرخه پرداخت

Course Purchase and Verification



دو relation پایدار هدف متفاوت دارند:

- TransactionsOnCourses: این checkout قصد خرید چه دوره‌هایی را داشت؟
- UsersOnCourses: اکنون کاربر مجاز به دسترسی به چه دوره‌هایی است؟

تنها تأیید موفق، IDهای courseهای تراکنش را به مالکیت منتقل می‌کند؛ در نتیجه checkout پرداخت‌نشده دسترسی نمی‌دهد؛ شاخه already-paid و کلید مرکب مالکیت، تکرار verify را تا حد زیادی idempotent می‌کند؛

به‌روزرسانی وضعیت تراکنش و ساخت مالکیت‌ها فعلاً دو عملیات جدا هستند و داخل یک prisma.\$transaction اجرا نمی‌شوند؛ خرابی بین این دو می‌تواند تراکنش paid بدون همه دسترسی‌ها بسازد؛ تا اتمیک‌شدن، فرایند reconciliation لازم است؛

اعتبارسنجی و خطاها

Zod ثبت‌نام، ورود، بازیابی/تغییر رمز، پروفایل و ساخت دوره را بررسی می‌کند؛ پرداخت و پارامترهای عددی URL دستی یا ضمنی validate می‌شوند؛

status	معنای رایج
200	موفقیت API؛ create/update/read فعلاً از 201 استفاده نمی‌کند
400	validation، سید/پرداخت نامعتبر، OTP اشتباه یا course lookup بد
403	ورود ناموفق یا access denied، با توجه به ایراد ترتیب middleware
404	دوره، ایمیل بازیابی یا تراکنش ناموجود
409	ایمیل ثبت‌نام تکراری
500	خطای مدیریت‌نشده؛ gateway، Prisma، یا server

گاهی object خام خطای Prisma در JSON قرار می‌گیرد؛ در production باید خطاها normalize شوند و جزئیات داخلی به client نرسند؛

امنیت

کنترل‌های موجود

- hash رمز با bcrypt؛
- cookie از نوع HTTP-only و در production دارای Secure؛
- بررسی امضای JWT و بارگذاری تازه user از دیتابیس؛
- بررسی ادمین از دیتابیس؛
- headerهای Helmet و allowlist تولید CORS؛
- محاسبه مبلغ پرداخت در سرور و verification سروربه‌سرور؛
- کلیدهای یکتا/مرکب برای جلوگیری از authority، subscriber، account و ownership تکراری؛
- پیام عمومی ورود برای کاهش account enumeration؛

الزامات عملیاتی

- JWT_SECRET با entropy بالا تولید و rotation آن برنامه‌ریزی شود؛
- env خارج از version control و secretها در secret manager نگهداری شوند؛
- production فقط روی HTTPS باشد؛
- دسترسی شبکه دیتابیس محدود، backup و در صورت نیاز TLS فعال باشد؛
- credentialهای sandbox و production زمین‌پال جدا باشند؛
- روی newsletter، OTP، register، login و payment rate limit اعمال شود؛
- payload کامل gateway و error حساس در production log نشود؛

ساخت، استقرار و عملیات

فرمان‌ها

فرمان	نتیجه
<code>npm run dev</code>	اجرای Nodemon روی TypeScript در development
<code>npm run build</code>	اجرای tsc و تولید dist
<code>npm start</code>	اجرای نسخه کامپایل شده در production
<code>npm test</code>	placeholder ناموفق؛ تستی وجود ندارد
<code>npmx prism generate</code>	بازسازی Prisma Client
<code>npmx prism migrate dev</code>	توسعه/اعمال migration محلی
<code>npmx prism migrate deploy</code>	اعمال migrationهای committ شده در استقرار
<code>npmx prism studio</code>	مرورگر محلی دیتابیس؛ نباید عمومی شود

پروژه هنگام تولید این مستند با `npm run build` با موفقیت کامپایل شد.

چک لیست production

۱. PostgreSQL و User کمدسترسی برنامه را بسازید؛
۲. secretهای الزامی و FRONTEND_URL صحیح را تنظیم کنید؛
۳. npm run dev را اجرا کنید؛
۴. npmx prism migrate deploy را در مرحله کنترل شده release اجرا کنید؛
۵. npm run build و سپس npm start را اجرا کنید؛
۶. برنامه را پشت reverse proxy/load balancer دارای HTTPS قرار دهید؛
۷. origin فرانت‌اند، domain cookie و routing را با مقادیر hard-coded production تطبیق دهید؛
۸. process supervisor، fog، health/readiness، metric، alert، backup و اضافه کنید؛
۹. یک خرید کم‌مبلغ production انجام دهید و ساخت ownership را بررسی کنید.

مقیاس پذیری

API به جز PostgreSQL و gateway خارجی Stateless است؛ چند replica می‌توانند دیتابیس مشترک داشته باشند و JWT/cookie نیازمند sticky session نیست؛ پیش از افزایش write scale، نهایی سازی پرداخت را atomid کنید و محدودیت connection یا pooler را در نظر بگیرید؛ هر process از Prisma connection pool خود استفاده می‌کند.

آزمون و رفع اشکال

Smoke test پیشنهادی

۱. ثبت نام و ورود و ذخیره شدن cookie را بررسی کنید؛
۲. کاربر آزمایشی را ادمین و یک دوره ایجاد کنید؛
۳. فهرست عمومی و جزئیات دوره را بخوانید؛
۴. در sandbox checkout را کامل و authority را verify کنید؛
۵. Transaction.isPaid=true و حضور دوره در /user/fetch-course را بررسی کنید؛
۶. verify را دوباره بفرستید و نبود ownership تکراری را تأیید کنید.

خطاهای رایج

نشانه	علت یا اقدام محتمل
برنامه پیش از listen خطا می‌دهد	merchant ID یا access token زربین‌پال وجود ندارد

Prisma وصل نمی‌شود	DATABASE_URL، شبکه، credential و migrationها را بررسی کنید
مرورگر login نمی‌ماند	credentials: {include: HTTPS, CORS, domain} و cookie را بررسی کنید
fallback توسعه به برنامه اشتباه می‌رود	FRONTEND_URL را تنظیم کنید؛ fallback پورت ۳۰۰۰ است
درگاه اشتباه sandbox/production باز می‌شود	فقط مقدار دقیق development sandbox را فعال می‌کند
حذف دوره 500 می‌دهد	احتمالاً join row وابسته وجود دارد
نتیجه admin user-course غلط است	ایراد شناخته‌شده userId/courseId
typeهای Prisma قدیمی‌اند	hpx prisma generate و سپس build اجرا شود

محدودیت‌ها و گام‌های بعدی

اولویت موارد زیر بر اساس ریسک و اثر است:

امنیت و صحت حیاتی

۱. بازیابی رمز را کامل کنید OTP؛ را با سرویس ایمیل ارسال و هرگز در JSON برنگردانید؛ expiry، hash، شمارنده تلاش و purpose ذخیره و پس از مصرف پاک کنید token؛ بازیابی نباید cookie ورود عمومی باشد؛
۲. برای JWT انقضا و lifecycle تعریف کنید rotation/revocation، expiresIn و invalidation پس از تغییر رمز؛
۳. ترتیب response در middleware را اصلاح کنید res.json(...).status(403) پیش از تعیین status پاسخ را می‌فرستد؛ باید res.status '403).json(...) باشد؛ مسیر user-not-found نیز همین مشکل را دارد؛ نتیجه nullable findUnique پیش از destructure بررسی شود؛
۴. نهایی‌سازی پرداخت را atomic کنید؛ تغییر تراکنش و اعطای مالکیت داخل prisma.\$transaction باشد و recovery محلی پس از موفقیت gateway تعریف شود؛
۵. ورودی پرداخت را سخت‌گیرانه validate کنید؛ آرایه غیرخالی و یکنای positive integer و رد کامل سید اگر حتی یک ID وجود ندارد؛
۶. جست‌وجوی دوره کاربر توسط ادمین را اصلاح کنید courseId؛ به جای userId map شود یا relational filter endpoint کاربر استفاده شود؛

بلوغ API و عملیات

۷. درباره verification حساب تصمیم بگیرید؛ verified/verificationKey را کامل و enforce یا حذف کنید؛
۸. برای درخواست‌های state-changing مبتنی بر cookie، CSRF protection، policy صریح same-site اضافه کنید؛
۹. rate، limit، envelope یکسان خطا، log، request ID، ساختاریافته و health/readiness اضافه کنید؛
۱۰. exception خام را به client ندهید و uniqueness/not-found/foreign-key را به 4xx امن map کنید؛
۱۱. img/link را URL-validate، previewLinks را در صورت نیاز محصول پشتیبانی و nullable بودن link را با API هماهنگ کنید؛
۱۲. pagination و response projection اضافه کنید؛ endpoint عمومی فعلاً تمام فیلدها، از جمله لینک کامل دوره، را می‌دهد؛
۱۳. تاریخچه/وضعیت تراکنش برای کاربر و reconciliation برای ادمین اضافه کنید؛
۱۴. برای خبرنامه unsubscribe و metadata رضایت اضافه کنید؛
۱۵. تست idempotency، unit، integration، authorization و npm test و CI را به آن متصل کنید؛

منابع مرجع سند؛ رفتار از app.ts، پوشه‌های schemas/، middleware/، src/ و utils/، فایل migration، schema.prisma، prisma/schema، SQL؛ package.json و tsconfig.json استخراج شده است؛ اگر کد اجرایی و سند متفاوت شدند، کد و migration اعمال‌شده حقیقت runtime هستند و سند باید در همان تغییر به روز شود؛